

Multiple Group Action Dlogs with(out) Precomputation

Alexander May[✉] and Massimo Ostuzzi[✉]

Ruhr-University Bochum, Bochum, Germany
alex.may@rub.de, massimo.ostuzzi@rub.de

Abstract. Let $\star : \mathcal{G} \times \mathcal{X} \rightarrow \mathcal{X}$ be the action of a group $\mathcal{G}$ of size $N = |\mathcal{G}|$ on a set $\mathcal{X}$. Let $y = g \star x \in \mathcal{X}$ be a *group action dlog* instance, where our goal is to compute the unknown group element $g \in \mathcal{G}$ from the known set elements $x, y \in \mathcal{X}$. The Galbraith-Hess-Smart (GHS) collision finding algorithm solves the group action dlog in $N^{\frac{1}{2}}$ steps with polynomial memory.

We show that group action dlogs are suitable for precomputation attacks. More precisely, for any s, t our precomputation algorithm computes within st steps a hint of space complexity s , which allows to solve any group action dlog in an online phase within t steps. A typical instantiation is $s = t = N^{\frac{1}{3}}$, which gives precomputation time $N^{\frac{2}{3}}$ and space $N^{\frac{1}{3}}$, and online time only $N^{\frac{1}{3}}$.

Moreover, we show that solving multiple group action dlog instances $y_1, \dots, y_m$ allows for speedups. Namely, our collision finding algorithm solves m group action dlogs in $\sqrt{m}N^{\frac{1}{2}}$ steps, instead of the straightforward $mN^{\frac{1}{2}}$ steps required for running m times GHS. Our multiple instance approach can be combined with our precomputations, allowing for a variety of tradeoffs.

Technically, our precomputation and multiple instance group action dlog attacks are adaptations of the techniques from the standard dlog setting in abelian groups. While such an adaptation seems natural, it is per se unclear which techniques transfer from the dlog to the more general group action dlog setting, for which $\mathcal{X}$ does not offer a group structure. Our algorithms have direct implications for all group action based cryptosystems, such as CSIDH and its variants. We provide experimental evidence that our techniques work well in the CSIDH setting.

Keywords: group actions · CSIDH · preprocessing · multi-instance dlogs · random walks.

1 Introduction

The invention of the discrete logarithm (dlog) based Diffie-Hellman (DH) key exchange in 1976 marks the birth of modern public-key cryptography. DH is nowadays used ubiquitously in practice. As a consequence, the discovery of Shor's polynomial time quantum algorithm for computing dlogs in any abelian

group [29] came as a shock to the cryptographic community, turning DH insecure in a quantum world.

While one would like to replace the group-based dlog problem by some quantum resistant problem, it is also desirable to retain the benefits of DH key exchange, such as small public keys, efficient computations, and compatibility with existing protocols.

The *group action dlog problem* evolved as a natural and elegant way to replace group-based dlogs, and group actions are the fruitful basis for a part of the new research area called isogeny-based cryptography [4,9,32,20] [19,11]. This gave birth to the key exchange protocol CSIDH [5], which provides an efficient instantiation of group actions, leading to a highly attractive and promising replacement of DH in a quantum world. The hope is that group action dlogs preserve all benefits from group based dlogs, while providing more security, especially against quantum computers.

Indeed, the group action dlog problem is a *hidden shift problem* [7], for which the best quantum algorithm by Kuperberg [22] has subexponential complexity. Nowadays, it is still an open question whether concrete instantiations of Kuperberg’s algorithm [25,3,6] pose threats to current group action dlog parameters.

Adaptation from Dlog Algorithms. Before considering a widespread replacement of DH by some group action based scheme like CSIDH, it is crucial to understand to which extent dlog attacks transfer to the group action dlog setting.

Using Pollard’s collision finding algorithm [27], we can compute a single dlog in any abelian group of size N in $O(N^{\frac{1}{2}})$ steps. The counterpart for group actions dlogs of instance size N is the algorithm of Galbraith, Hess and Smart [15,14], likewise requiring $O(N^{\frac{1}{2}})$ steps.

Other dlog algorithms do not transfer naturally. One crucial example is again Shor’s polynomial time dlog algorithm, for which its group action counterpart Kuperberg’s algorithm requires subexponential time. A second example is the Silver-Pohlig-Hellman algorithm [26], that exploits smoothness of the group order, for which currently no counterpart is believed to exist in the group action dlog setting [5].

Therefore it is of great importance to understand which dlog algorithms can be transferred into the group action setting at all.

Dlog Precomputation and Multi-Instances. The discrete log setting has the nice feature that one can standardize groups that are believed to be especially efficient, and for which the dlog problem is considered hard. Examples are the current elliptic curve standards, like NIST P-256. These standardized groups provide an advantage over RSA based key exchange, for which users may generate insecure instances [18].

An analogous property holds for group action based schemes. For instance CSIDH-512 provides an efficient systemwide instantiation of a group action that all users are supposed to use securely. In fact, it seems that the group action setting is even more restrictive, in the sense that it is harder to find suitable instantiations that are both efficient and secure [5].

The drawback of systemwide instantiations is that they are attractive targets for powerful adversaries. It appears plausible that a large scale adversary, such as a national state agency, has the capabilities to perform a heavy precomputation. For a cryptographic (group action) standard that is used by billions of devices, such a precomputation may run over several years to produce some hint. The hint in turn could allow the large scale adversary for significantly more efficient (group action) dlog computations in an online phase. In the dlog setting, precomputations have been studied by [1,8].

Moreover, large scale adversaries interested in mass surveillance of users not only desire to compute a single (group action) dlog, but aim to amortize their costs for recovering a plethora of cryptographic keys. In the dlog setting, it is known that m dlogs can be computed in time $\sqrt{m}N^{\frac{1}{2}}$ [21,13,34], instead of the naive $mN^{\frac{1}{2}}$ by applying Pollard's algorithm m times, thereby amortizing the attack costs.

Our main contribution is to transfer both the precomputation and the multi-instance attacks from the dlog to the group action dlog setting. To this end, let us work out in the following the similarities (and limitations) of both settings in a bit more detail.

Discrete Logarithms. Let us recap the discrete logarithm in a finite cyclic group H generated by $h \in H$, in which we denote the group operation as multiplication. We write $H = \langle h \rangle = \{h, h^2, \dots, h^{\text{ord}(h)}\}$, with $\text{ord}(h) = |H|$. We denote the integers modulo $|H|$ by $\mathbb{Z}_{|H|} := \mathbb{Z}/|H|\mathbb{Z}$.

Let us consider the exponentiation map $\varphi_h : \mathbb{Z} \rightarrow H, v \mapsto h^v$. Notice that $\lambda = \ker(\varphi_h) = |H|\mathbb{Z}$ is a 1-dimensional lattice in $\mathbb{Z}$. Therefore, the following map is a bijection

$$f_h : \mathbb{Z}/\lambda \rightarrow H, \quad v \mapsto h^v.$$

Let $y = h^v = f_h(v)$. The *discrete logarithm (dlog) problem in H* is to invert f_h on y , namely to compute the unique $v = f_h^{-1}(y) \bmod \lambda$.

In *group action* terminology, for our discrete logarithm problem the group $\mathcal{G} := \mathbb{Z}/\lambda = \mathbb{Z}_{|H|}$ acts on the *group* H . The group structure of H itself is exploited in many algorithms, such as Shor's algorithm [29] and Pollard's Rho algorithm [27].

Pollard Rho uses the technique of *collision finding*. Let $y = h^v$ and define

$$f_{h,y} : \mathcal{G} \times \mathcal{G} \rightarrow H, \quad (x, z) \mapsto h^x y^z.$$

Moreover, suppose $(x, z) \neq (x', z')$ is a collision in $f_{h,y}$, namely $f_{h,y}(x, z) = f_{h,y}(x', z')$. We have

$$f_{h,y}(x, z) = h^x y^z = h^{x+yz},$$

and likewise

$$f_{h,y}(x', z') = h^{x'} y^{z'} = h^{x'+yz'}.$$

We conclude that the collision $(x, z), (x', z')$ directly yields the discrete logarithm as $v = \frac{x-x'}{z'-z} \bmod \lambda$, provided that $z' - z$ is invertible modulo λ . Notice that our reasoning relies on H 's group structure.

Group Action Dlogs. Let us now introduce the *group action discrete logarithm* (GA-dlog) problem, and discuss its similarities and differences to the ordinary discrete logarithm problem in a group H .

Let $\mathcal{X}$ be a set, *without any group structure*. Let $x \in \mathcal{X}$ be a distinguished element, called the *origin*, that plays the role of a generator of $\mathcal{X}$. Namely, we let some finite abelian group $\mathcal{G}$ act on $\mathcal{X}$ with $\star$ via the origin x , such that $\{g \star x \mid g \in \mathcal{G}\} = \mathcal{X}$. Notice that our definition of $\mathcal{X}$ already implies $N = |\mathcal{G}| \geq |\mathcal{X}|$, but we will furthermore require that $|\mathcal{X}| = |\mathcal{G}|$.

Let us assume that $\mathbf{g} = \{g_1, \dots, g_n\}$ is a finite set of generators for $\mathcal{G}$, denoted $\mathcal{G} = \langle \mathbf{g} \rangle = \langle g_1, \dots, g_n \rangle$. Moreover, for any integer vector $\mathbf{v} = (v_1, \dots, v_n) \in \mathbb{Z}^n$ we write $\mathbf{g}^{\mathbf{v}} = g_1^{v_1} \cdot \dots \cdot g_n^{v_n}$.

Let us consider the exponentiation map $\varphi_{\mathbf{g}} : \mathbb{Z}^n \rightarrow \mathcal{G}, \mathbf{v} \mapsto \mathbf{g}^{\mathbf{v}}$. Notice that $\Lambda = \ker(\varphi_{\mathbf{g}})$ is an n -dimensional lattice in $\mathbb{Z}^n$. The following map is a bijection

$$f_{\mathbf{g},x} : \mathbb{Z}^n / \Lambda \rightarrow \mathcal{X}, \quad \mathbf{v} \mapsto \mathbf{g}^{\mathbf{v}} \star x. \quad (1)$$

Let $y = \mathbf{g}^{\mathbf{v}} \star x = f_{\mathbf{g},x}(\mathbf{v})$. Then the *GA-dlog problem in $\mathcal{X}$* is to invert $f_{\mathbf{g},x}$ on y , namely to compute the unique $\mathbf{v} = f_{\mathbf{g},x}^{-1}(y) \bmod \Lambda$, that in turn represents the group element $\mathbf{g}^{\mathbf{v}} \in \mathcal{G}$.

Notice that the missing group structure of $\mathcal{X}$ prevents a straight-forward adaptation of Pollard's collision finding technique, as well as an adaptation of Shor's quantum dlog algorithm.

Our Contributions. Let $y = \mathbf{g}^{\mathbf{v}} \star x$ be a GA-dlog problem for a group $\mathcal{G} = \langle \mathbf{g} \rangle$ of size $N = |\mathcal{G}|$. We define the two functions

$$\begin{aligned} f_{\mathbf{g},x} : \mathbb{Z}^n / \Lambda &\rightarrow \mathcal{X}, & \mathbf{v} &\mapsto \mathbf{g}^{\mathbf{v}} \star x, \\ f_{\mathbf{g},y} : \mathbb{Z}^n / \Lambda &\rightarrow \mathcal{X}, & \mathbf{v} &\mapsto \mathbf{g}^{\mathbf{v}} \star y, \end{aligned}$$

Let $(\mathbf{v}_1, \mathbf{v}_2)$ be a collision of $f_{\mathbf{g},x}, f_{\mathbf{g},y}$, namely $f_{\mathbf{g},x}(\mathbf{v}_1) = f_{\mathbf{g},y}(\mathbf{v}_2)$. Then we have

$$\begin{aligned} f_{\mathbf{g},x}(\mathbf{v}_1) &= \mathbf{g}^{\mathbf{v}_1} \star x, \text{ and} \\ f_{\mathbf{g},y}(\mathbf{v}_2) &= \mathbf{g}^{\mathbf{v}_2} \star y = \mathbf{g}^{\mathbf{v}_2} \star (\mathbf{g}^{\mathbf{v}} \star x) \stackrel{(*)}{=} (\mathbf{g}^{\mathbf{v}_2 + \mathbf{v}}) \star x, \end{aligned}$$

where $(*)$ holds by a group action property called *compatibility*. By injectivity of the map in Equation (1), we obtain the GA-dlog as $\mathbf{v} = \mathbf{v}_1 - \mathbf{v}_2 \bmod \Lambda$.

Precomputation. Observe that the function $f_{\mathbf{g},x}$, as opposed to $f_{\mathbf{g},y}$, solely depends on the group action defined via $(\mathbf{g}, x)$, but not on a GA-dlog instance y . Therefore, we call $f_{\mathbf{g},x}$ *instance-independent*, while $f_{\mathbf{g},y}$ is called *instance-dependent*. The instance-independence of $f_{\mathbf{g},x}$ allows us to precompute via $(\mathbf{g}, x)$ alone a hint for the group action. On obtaining a concrete GA-dlog instance y , one can then use the hint to compute more efficiently a collision via $f_{\mathbf{g},y}$.

More concretely, for $f_{\mathbf{g},x}$ we precompute s instance-independent random walks, each of them of length t . This requires time st and space s , by storing only their endpoints, which then serve as our hint. Let us look at a typical parameter choice $s = t = N^{\frac{1}{3}}$. Then precomputation requires time $st = N^{\frac{2}{3}}$ and space only $s = N^{\frac{1}{3}}$.

We show that our precomputation already touches roughly $st = N^{\frac{2}{3}}$ points in $\mathcal{X}$. Thus, we expect that any instance-dependent walk with $f_{\mathbf{g},y}$ of length $t = N^{\frac{1}{3}}$ collides with one of these points, thereby yielding a solution to the GA-dlog problem.

Multiple Instances. Let $y_1, \dots, y_m$ be m GA-dlog instances. Solving all instances via the Galbraith-Hess-Smart algorithm requires time $mN^{\frac{1}{2}}$. We show that one can solve *all instances* in time only $\sqrt{m}N^{\frac{1}{2}}$, thereby saving a $\sqrt{m}$ -factor and solving a single instance in amortized (over all m instances) cost $(\frac{N}{m})^{\frac{1}{2}}$.

Precomputation and Multiple Instances. The idea of precomputation allows for a combination with the multi-instance setting. Namely, one first precomputes an *instance independent* structure. On obtaining m instances $y_1, \dots, y_m$, one then lets m instance-dependent random walks $f_{\mathbf{g},y_1}, \dots, f_{\mathbf{g},y_m}$ collide into the precomputed points.

This allows for various tradeoffs. For instance, one may precompute in time $mN^{\frac{2}{3}}$ a structure of size $m^2N^{\frac{1}{3}}$, which then in turn allows to solve *all m instances* in time only $N^{\frac{1}{3}}$.

Precomputation includes Multi-Instance. Technically, we apply and transfer the precomputation dlog framework by Corrigan-Gibbs and Kogan [8]. This framework was already successfully applied to transfer precomputation and multi-instances from dlogs to the Legendre PRF setting [23]. In fact, our analysis closely follows the reasoning for the precomputation setting in [8,23].

However, for the multi-instance setting *without* precomputation we slightly deviate from [23]. Namely, we observe that the multi-instance setting (without precomputation) is a special case of the multi-instance precomputation setting. While we will see that our observation is somewhat straight-forward, to the best of our knowledge it has been overlooked in the cryptographic literature so far. Despite its triviality, our observation is also a bit counter-intuitive, probably explaining why it slipped through. Indeed, why should an algorithm *without* precomputation drop out from a precomputation scenario, which is usually expected to perform a heavy initial precomputation phase?

In fact, the trick is to perform only a light precomputation, balancing the cost between precomputation and online phase. This implies that we do not have to separate between precomputation and online phase any longer, thereby omitting precomputation altogether.

As a consequence of our observation, we obtain for free an appealingly simple multi-instance algorithm with a clean and elegant analysis for GA-dlogs. The same is true for ordinary dlogs, for which we explicitly provide a multi-instance algorithm from its precomputation algorithm. As opposed to other

multi-instance dlog algorithms, our approach does not require the use of distinguished point techniques [21,13], or heavy graph theory analysis [13]. It is easy to see that our technique transfers to other settings as well, as e.g. to multi-instance Legendre PRF [23] or to multi-user Even-Mansour [13].

Lower Bounds and Optimality. Our algorithms are fully generic. Namely, they work for any group action and only require equality testing of elements. Any improvement on generic GA-dlog algorithms would immediately yield an improvement on generic algorithms for dlogs. However, this would contradict the optimality of current generic dlog algorithms, as shown by the lower bounds of Yun [34] and Corrigan-Gibbs, Kogan [8]. We achieve upper bounds matching the lower bounds in [34,8], showing optimality of our generic algorithms.

The Role of the Lattice Λ . Notice that in an ordinary discrete log setting over a group H , we assume the group order $|H|$ to be known. This helps during random walks to update all discrete logs modulo $|H|$, thereby controlling their sizes. In elliptic curve groups the knowledge of $|H|$ is a reasonable assumption, since it can be computed via Schoof’s algorithm [28] in polynomial time.

Just as we assume knowledge of $|H|$ in the dlog setting, we assume in the group action dlog setting knowledge of a basis of Λ as input to our algorithms. Analogously, this allows us during random walks to update all GA-dlogs modulo Λ . In the CSIDH group action setting, a basis of Λ can be computed in quantum polynomial time [17]. Classically, we may compute such a basis in subexponential time via the algorithm of [16]. For instance, for CSIDH-512 a basis has been computed in [2]. Strictly speaking, our algorithms also work without knowing Λ , but operating with the output GA-dlog without any reduction modulo Λ might take exponential time, depending on the specific group action. For example, CSIDH group action evaluation using Vélú’s formulae takes exponential time in the size of the coordinates of $\mathbf{v} \in \mathbb{Z}^n$. However, recent developments [19,24] allow to evaluate the class group action in polynomial time with no lattice reduction, leveraging the attack on SIDH in a constructive way.

Provability. We prove correctness, complexity and success probability of our algorithms without any heuristic assumptions, solely relying on a PRF realizing a mapping $\mathcal{X} \rightarrow \mathbb{Z}^n/\Lambda$ that we need for the analysis of our random walks. In our experiments for CSIDH, we show that we can easily realize a random mapping for CSIDH that works well in practice (although not being a PRF).

Notice that, by our mapping $\mathcal{X} \rightarrow \mathbb{Z}^n/\Lambda$, a single step in our random walks does not coincide with moving to a random neighbor in the so-called isogeny graph (as in other algorithms like [14,12]), but we rather randomly jump in $\mathcal{X}$. This has the advantage that we do not have to care about the isogeny graph’s mixing properties. On the downside, a single step in our walks is computationally more expensive. In our approach, we prefer clarity of exposition and provability over potential implementation practicality.

Implications for CSIDH-512. CSIDH-512 works with elliptic curves over $\mathbb{F}_p$ with 512-bit prime p , leading to $N = |\mathcal{G}| = |\mathcal{X}|$ of 256 bit size. Therefore, CSIDH-512 offers 128 bit security against the Delfs-Galbraith algorithm. Using our precomputation algorithm with parameter choice $s = t = N^{\frac{1}{3}}$ would lead to 171 bit of precomputations for a large scale adversary, resulting in a hint of size 85 bits. Such a hint would allow to solve single GA-dlogs within only 2^{85} bit operations. Since such a large scale attacker seems unrealistic, our precomputation attack currently does not directly affect the CSIDH-512 security level.

Organization of the paper. In Section 2, we define group action dlogs. Section 3 is devoted to our GA-dlog algorithm for a single instance with precomputation, which we generalize to multiple instances in Section 4. In Section 5, we show that a multi-instance GA-dlog without precomputation follows as a special case from the algorithm in Section 4. In Section 6, we provide as a further application and, for completeness, the analogous multi-instance dlog algorithm. In Section 7, we show that our precomputation GA-dlog algorithm works well in practice for small-scale parameter sets of CSIDH.

Implementation. The code for our CSIDH experiments is available at https://github.com/maxostuzzi/precomputation_attack.

2 Preliminaries

Let us define a group action and the discrete logarithm problem for group actions.

Definition 1 (Group action). Let $(\mathcal{G}, \cdot)$ be a multiplicative group, and let $\mathcal{X}$ be a set. The map

$$\star: \mathcal{G} \times \mathcal{X} \rightarrow \mathcal{X}$$

is called a group action of $\mathcal{G}$ on $\mathcal{X}$, denoted $(\mathcal{G}, \mathcal{X}, \star)$, if it satisfies the properties

1. Identity: $1 \star x = x$, for all $x \in \mathcal{X}$.
2. Compatibility: $(g \cdot h) \star x = g \star (h \star x)$, for all $g, h \in \mathcal{G}$ and $x \in \mathcal{X}$.

For efficient computations, we require that we can compactly represent $\mathcal{G}$ and $\mathcal{X}$. To this end we assume that $\mathcal{G}$ is finite and generated by $\mathbf{g} = \{g_1, \dots, g_n\}$, denoted as $\mathcal{G} = \langle \mathbf{g} \rangle$. Moreover, we let $x \in \mathcal{X}$ be a distinguished element called *origin*, satisfying

$$\mathcal{X} = \{g \star x \mid g \in \mathcal{G}\} \quad \text{and} \quad |\mathcal{X}| = |\mathcal{G}| := N. \quad (2)$$

In other words, the map $\mathcal{G} \rightarrow \mathcal{X}, g \mapsto g \star x$ is a bijection. Group actions satisfying Equation (2) are called *regular* in the literature.

Definition 2 (Representation). Let $\mathbf{v} = (v_1, \dots, v_n) \in \mathbb{Z}^n$. We denote $\mathbf{g}^{\mathbf{v}} := g_1^{v_1} \cdot \dots \cdot g_n^{v_n} \in \mathcal{G}$. For some group element $\mathbf{g}^{\mathbf{v}} \in \mathcal{G}$, we call its exponent vector $\mathbf{v}$ a representation.

Consider the exponentiation map $\phi_{\mathbf{g}}: \mathbb{Z}^n \rightarrow \mathcal{G}, \mathbf{v} \rightarrow \mathbf{g}^{\mathbf{v}}$. It is a surjective map and its kernel $\Lambda := \ker(\phi_{\mathbf{g}})$ is an n -dimensional lattice in $\mathbb{Z}^n$. Therefore, the map $\mathbb{Z}^n/\Lambda \rightarrow \mathcal{G}$ is an isomorphism, and every element in $\mathcal{G}$ has a representation that is unique modulo Λ . To simplify the notation, we are going to write simply $\mathbf{v} \in \mathbb{Z}^n/\Lambda$ for the equivalence class of the vector $\mathbf{v} \in \mathbb{Z}^n$.

Definition 3 (GA-dlog). Let $(\mathcal{G}, \mathcal{X}, \star)$ be regular group action with generators $\mathbf{g}$ and origin x , namely

$$\mathcal{G} = \langle \mathbf{g} \rangle = \langle g_1, \dots, g_n \rangle \cong \mathbb{Z}^n/\Lambda, \quad \mathcal{X} = \{g \star x \mid g \in \mathcal{G}\} \quad \text{and} \quad |\mathcal{G}| = |\mathcal{X}|.$$

In the group action discrete logarithm (GA-dlog) problem, the goal is to find on input $(\mathbf{g}, x, y) \in \mathcal{G}^n \times \mathcal{X}^2$ the unique representation $\mathbf{v} \in \mathbb{Z}^n/\Lambda$ satisfying $y = \mathbf{g}^{\mathbf{v}} \star x$.

3 Solving GA-dlogs with Precomputation

High-Level Description. Let us first give a high-level description of our group action dlog algorithm with precomputation, see also Figure 1.

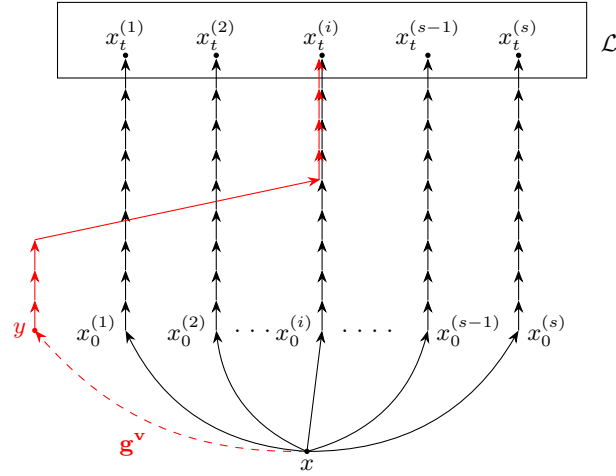


Fig. 1: Collision finding for GA-dlog instance $y = \mathbf{g}^{\mathbf{v}} \star x$ with precomputation.

Our *precomputation phase* is *instance-independent* and solely relies on the parameters $(\mathbf{g}, x)$ defining the group action. We start s instance-independent random walks $W^{(1)}, \dots, W^{(s)}$ on $\mathcal{X}$, and store only their endpoints in a list $\mathcal{L}$.

In the *online phase*, we receive a group action dlog instance $y = \mathbf{g}^{\mathbf{v}} \star x$. We then start an *instance-dependent* random walk, and let it collide into one of the walks $W^{(1)}, \dots, W^{(s)}$. The collision is identified via the stored endpoints. In the following we show that a collision immediately yields the desired GA-dlog $\mathbf{v}$.

Precomputation Phase. Let us describe a single random walk $W^{(i)}$ from the precomputation phase. To this end, we choose a random $\mathbf{w}_0^{(i)} \in \mathbb{Z}^n/\Lambda$ that defines a random starting point $x_0^{(i)} := \mathbf{g}^{\mathbf{w}_0^{(i)}} \star x \in \mathcal{X}$.

Let $h : \mathcal{X} \rightarrow \mathbb{Z}^n/\Lambda$ be a pseudorandom function (PRF), which allows us to define our random walk as

$$x_j^{(i)} := \mathbf{g}^{h(x_{j-1}^{(i)})} \star x_{j-1}^{(i)} \quad \text{for } j \geq 1. \quad (3)$$

In Section 7, we show how to instantiate h for the group action in CSIDH, as an example in a concrete setting. Moreover, let us define

$$\begin{aligned} \mathbf{w}_j^{(i)} &:= h(x_{j-1}^{(i)}) + \mathbf{w}_{j-1}^{(i)} \\ &= h(x_{j-1}^{(i)}) + h(x_{j-2}^{(i)}) + \mathbf{w}_{j-2}^{(i)} = \dots = \mathbf{w}_0^{(i)} + \sum_{k=0}^{j-1} h(x_k^{(i)}) \quad \text{for } j \geq 1. \end{aligned} \quad (4)$$

Notice that, by the definition of our random walk and by the Compatibility property of Definition 1, we have

$$x_j^{(i)} = \mathbf{g}^{h(x_{j-1}^{(i)})} \star (\mathbf{g}^{h(x_{j-2}^{(i)})} \star \dots \star (\mathbf{g}^{h(x_0^{(i)})} \star (\mathbf{g}^{\mathbf{w}_0^{(i)}} \star x)) \dots) = \left(\mathbf{g}^{\mathbf{w}_0^{(i)}} \prod_{k=0}^{j-1} \mathbf{g}^{h(x_k^{(i)})} \right) \star x.$$

Therefore, we conclude that $x_j^{(i)}$ has GA-dlog $\mathbf{w}_j^{(i)}$, as defined in Eq. (4).

For each of the s random walks $W^{(i)}$ of length t , we store only their endpoint $x_t^{(i)}$ together with its GA-dlog $\mathbf{w}_t^{(i)}$.

The resulting precomputation is detailed in PRECOMPUTE-GA (Algorithm 1).

Online Phase. Let $y = \mathbf{g}^{\mathbf{v}} \star x$ be a GA-dlog instance. We start a random walk W as defined in Equation (3) from the starting point $x_0 := y$, see also Figure 1. As in the precomputation phase, we keep track of the GA-dlog. Namely, the random walk point x_j after j steps has GA-dlog

$$\mathbf{v} + \sum_{k=0}^{j-1} h(x_k) = \mathbf{v} + \mathbf{w}_j. \quad (5)$$

However, notice that $\mathbf{v}$ is the desired *unknown* GA-dlog of y , therefore we only store the value $\mathbf{w}_j = \sum_{k=0}^{j-1} h(x_k)$.

Eventually, our walk W collides into one of the precomputed walks. Let $W^{(\ell)}$ be the colliding walk. Let us first show that once W collides into $W^{(\ell)}$, both walks subsequently visit the same points.

Colliding walks stay together. Let $x_j = x_k^{(\ell)}$ be the first collision between W and $W^{(\ell)}$. Then, by Equation (3), we have

$$x_{j+1} = \mathbf{g}^{h(x_j)} \star x_j = \mathbf{g}^{h(x_k^{(\ell)})} \star x_j = \mathbf{g}^{h(x_k^{(\ell)})} \star x_k^{(\ell)} = x_{k+1}^{(\ell)}.$$

Algorithm 1: PRECOMPUTE-GA

Input: group action parameters $(\mathbf{g}, x) \in \mathcal{G}^n \times \mathcal{X}$, $N := |\mathcal{G}|$, basis for $\Lambda \subseteq \mathbb{Z}^n$,
PRF $h : \mathcal{X} \rightarrow \mathbb{Z}^n / \Lambda$

Output: hint list $\mathcal{L}$ of endpoints/GA-dlogs $(x_t^{(i)}, \mathbf{w}_t^{(i)}) \in \mathcal{X} \times \mathbb{Z}^n / \Lambda$

- 1 Choose $s, t \in \mathbb{N}$ s.t. $4st^2 \leq N$ // E.g. $s, t = \lfloor \frac{1}{2} N^{\frac{1}{3}} \rfloor$
- 2 $\mathcal{L} \leftarrow \emptyset$
- 3 **for** $i = 1, \dots, s$ // Compute random walks $W^{(1)}, \dots, W^{(s)}$.
- 4 **do**
- 5 Choose a random $\mathbf{w}_0^{(i)} \in \mathbb{Z}^n / \Lambda$.
- 6 Let $x_0^{(i)} := \mathbf{g}^{\mathbf{w}_0^{(i)}} \star x \in \mathcal{X}$. // Randomized starting point.
- 7 **for** $j = 1, \dots, t$ // Each walk $W^{(i)}$ has length t .
- 8 **do**
- 9 Let $x_j^{(i)} := \mathbf{g}^{h(x_{j-1}^{(i)})} \star x_{j-1}^{(i)} \in \mathcal{X}$ and $\mathbf{w}_j^{(i)} := h(x_{j-1}^{(i)}) + \mathbf{w}_{j-1}^{(i)} \bmod \Lambda$.
- 10 $\mathcal{L} \leftarrow \mathcal{L} \cup \left\{ (x_t^{(i)}, \mathbf{w}_t^{(i)}) \right\}$ // Store endpoint/GA-dlog in hint $\mathcal{L}$.
- 11 Sort $\mathcal{L}$ by first entry. // Allows for binary search in $\mathcal{L}$.
- 12 **return** $\mathcal{L}$

Inductively, we obtain $x_{j+i} = x_{k+i}^{(\ell)}$ for all $i \geq 0$, which means that the walks stay together, see also Figure 1. As a consequence, the online phase walk W will eventually reach $W^{(\ell)}$'s stored endpoint $x_t^{(\ell)}$, together with its GA-dlog $\mathbf{w}_t^{(\ell)}$. It remains to show that the tuple $(x_t^{(\ell)}, \mathbf{w}_t^{(\ell)})$ reveals the solution of the GA-dlog instance y .

Endpoints solve GA-dlog. Let $x_j = x_t^{(\ell)}$ be the colliding endpoints of W and $W^{(\ell)}$. By Equation (5) and Equation (4), their GA-dlogs are

$$\mathbf{v} + \mathbf{w}_j, \quad \text{respectively} \quad \mathbf{w}_t^{(\ell)} = \mathbf{w}_0^{(\ell)} + \sum_{k=0}^{t-1} h(x_k^{(\ell)}).$$

Since $\mathbf{v} + \mathbf{w}_j = \mathbf{w}_t^{(\ell)} \bmod \Lambda$, we obtain the desired GA-dlog of y as

$$\mathbf{v} = \mathbf{w}_t^{(\ell)} - \mathbf{w}_j \bmod \Lambda.$$

The resulting online phase is detailed in ONLINE-GA-DLOG (Algorithm 2).

Remark 1. Notice that ONLINE-GA-DLOG fails if we do not collide within $2t$ steps into one of the precomputed walks $W^{(1)}, \dots, W^{(s)}$. In practice, one may then restart *only the online walk* with a fresh re-randomized starting point $x_0 := \mathbf{g}^{\mathbf{w}_0} \star y$, for some random $\mathbf{w}_0 \in \mathbb{Z}^n / \Lambda$. This amplifies the success probability arbitrarily close to 1, see also our experiments for CSIDH in Section 7. In subsequent sections, we will use re-randomized starting points to allow for success amplification.

Algorithm 2: ONLINE-GA-DLOG

Input: $(\mathbf{g}, x, y = \mathbf{g}^{\mathbf{v}} \star x) \in \mathcal{G}^n \times \mathcal{X}^2$, $N := |\mathcal{G}|$, basis for $\Lambda \subseteq \mathbb{Z}^n$,
precomputed hint $\mathcal{L} \in (\mathcal{X} \times \mathbb{Z}^n/\Lambda)^s$, t , PRF $h : \mathcal{X} \rightarrow \mathbb{Z}^n/\Lambda$

Output: GA-dlog $\mathbf{v} \in \mathbb{Z}^n/\Lambda$ or FAIL

- 1 Let $\mathbf{w}_0 = 0^n$ and $x_0 := y$.
- 2 **for** $j = 1, \dots, 2t$ // 2t-step walk W
- 3 **do**
- 4 Let $x_j := \mathbf{g}^{h(x_{j-1})} \star x_{j-1} \in \mathcal{X}$ and $\mathbf{w}_j := h(x_{j-1}) + \mathbf{w}_{j-1} \bmod \Lambda$.
- 5 **if** $(x_j, \mathbf{w}_t^{(\ell)}) \in \mathcal{L}$ for some $\ell \in \{1, \dots, s\}$ // Endpoint in $\mathcal{L}$?
- 6 **then**
- 7 **return** $\mathbf{v} := \mathbf{w}_t^{(\ell)} - \mathbf{w}_j \bmod \Lambda$
- 8 **return** FAIL

Theorem 1. *Let $(\mathcal{G}, \mathcal{X}, \star)$ be a regular group action with $N = |\mathcal{G}| = |\mathcal{X}|$. For any choice of $s, t \in \mathbb{N}$ with $4st^2 \leq N$, PRECOMPUTE-GA-DLOG (Algorithm 1) precomputes within st steps a hint $\mathcal{L}$ of size $\tilde{O}(s)$. Using $\mathcal{L}$, ONLINE-GA-DLOG (Algorithm 2) solves a GA-dlog instance within $O(t)$ steps with success probability $\Omega\left(\frac{st^2}{N}\right)$.*

Proof. PRECOMPUTE-GA computes s random walks of length t in a total of st steps. Since we store only their endpoints/GA-dlogs in our hint list $\mathcal{L}$, our memory requirement is $\tilde{O}(s)$. ONLINE-GA-DLOG performs at most $2t = O(t)$ steps to find a collision. If ONLINE-GA-DLOG collides within its first t steps into some precomputed walk $W^{(i)}$, then it reaches within its subsequent t steps an endpoint in $\mathcal{L}$. By the previous discussion, the endpoint yields the solution to the GA-dlog instance $y = \mathbf{g}^{\mathbf{v}} \star x$.

It remains to show that ONLINE-GA-DLOG succeeds to collide within t steps into a precomputed walk with probability $\Omega(\frac{st^2}{N})$. To this end, we show that PRECOMPUTE-GA touches with probability at least $\frac{1}{2}$ within its st steps at least $st/2$ different elements in $\mathcal{X}$.

Let X_i be a random variable for the number of elements in $\mathcal{X}$ newly visited by walk W_i . Let $X = \sum_{i=1}^s X_i \leq st$. Using the randomness property of our PRF h , Bernoulli's inequality, and $4st^2 \leq N$, we show that each walk $W^{(i)}$ newly visits the maximum number of $X_i = t$ new elements from $\mathcal{X}$ with probability at least

$$\mathbf{P}[X_i = t] \geq \left(\frac{N - st}{N}\right)^t = \left(1 - \frac{st}{N}\right)^t \geq 1 - \frac{st^2}{N} \geq \frac{3}{4}.$$

Hence, we have on expectation $\mathbf{E}[X_i] \geq \frac{3t}{4}$ newly visited $\mathcal{X}$ -elements for each precomputed walk and, by linearity of expectation,

$$\mathbf{E}[X] = \sum_{i=1}^s \mathbf{E}[X_i] \geq \frac{3}{4}st.$$

Using Markov's inequality, we obtain

$$\mathbf{P} \left[X < \frac{st}{2} \right] \leq \mathbf{P} \left[st - X \geq \frac{st}{2} \right] \leq \frac{st - \mathbf{E}[X]}{\frac{st}{2}} \leq \frac{1}{2}.$$

Hence, with probability at least $\frac{1}{2}$ PRECOMPUTE-GA visits within its st steps at least $X \geq st/2$ distinct elements in $\mathcal{X}$.

Let C_t be the event that ONLINE-GA-DLOG collides within the first t steps with one of the precomputed $\mathcal{X}$ -elements, which is sufficient for ONLINE-GA-DLOG to solve the GA-dlog problem.

Using $1 - x \leq e^{-x}$ and $1 - e^{-x} \geq x/2$ for $x \leq 1$, we obtain

$$\mathbf{P} \left[C_t \mid X \geq \frac{st}{2} \right] \geq 1 - \left(1 - \frac{st}{2N} \right)^t \geq 1 - e^{-\frac{st^2}{2N}} \geq \frac{st^2}{4N}.$$

Hence, ONLINE-GA-DLOG succeeds with probability at least

$$\mathbf{P} \left[C_t \cap \left(X \geq \frac{st}{2} \right) \right] = \mathbf{P} \left[X \geq \frac{st}{2} \right] \cdot \mathbf{P} \left[C_t \mid X \geq \frac{st}{2} \right] \geq \frac{st^2}{8N} = \Omega \left(\frac{st^2}{N} \right). \quad \square$$

4 Solving Multiple GA-Dlogs with Precomputation

If an attacker makes the effort of a heavy precomputation for some group action instance $(\mathcal{G}, \mathcal{X}, \star)$, then the goal is usually not to online tackle just a single GA-dlog instance, but rather to solve a large quantity of GA-dlog instances simultaneously.

Let $y_1 = \mathbf{g}^{\mathbf{v}_1} \star x, \dots, y_m = \mathbf{g}^{\mathbf{v}_m} \star x$ be m GA-dlog instances. Our goal is to solve *all* instances. In a nutshell, a slightly heavier precomputation pays off in amortizing the cost over all m instances. As an example, we show in the following that a precomputation in time $m^2 N^{\frac{2}{3}}$ that produces a hint of size $m^2 N^{\frac{1}{3}}$ allows to solve *all* m instances in total time only $N^{\frac{1}{3}}$. Various other tradeoffs between precomputation time, hint size and online time are possible.

High-Level Idea. The multiple instance setting with precomputation is a natural generalization of the single instance setting from Section 3. Again, we precompute s random walks $W^{(1)}, \dots, W^{(s)}$ in $\mathcal{X}$, and store only their endpoints, together with their GA-dlogs. During the online phase we then let all m online walks $\overline{W}^{(1)}, \dots, \overline{W}^{(m)}$ collide into one of the precomputed walks. However, in order to obtain constant success probability for solving all m GA-dlog instances, we have to adjust the lengths of all walks during precomputation and online phase accordingly. Details of this adjustment follow.

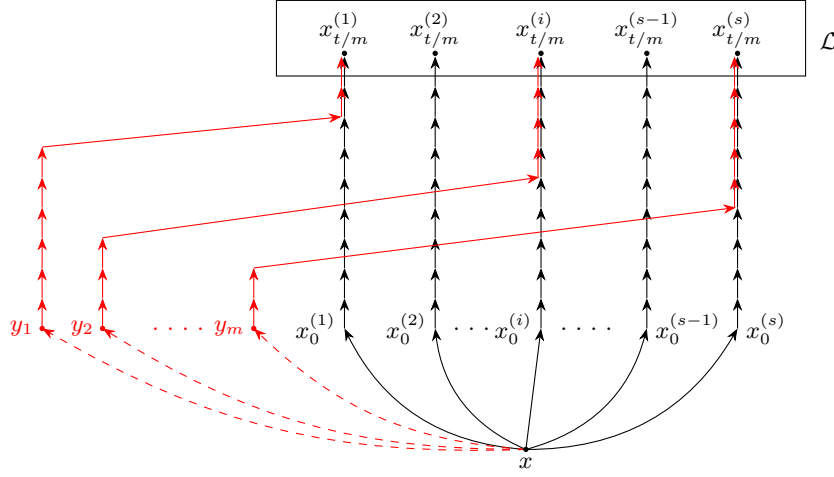


Fig. 2: Collision finding for multiple GA-dlogs with precomputation.

Generalization to the Multi-Instance Setting. We provide our algorithms PRECOMPUTE-MULT-GA, ONLINE-MULT-GA-DLOG and ALL-GA-DLOG in Algorithms 3 to 5. We advise the reader to compare the first two to PRECOMPUTE-GA and ONLINE-GA-DLOG (Algorithms 1 and 2) from Section 3.

While in the single instance setting we obtained constant success probability by the parameter choice $st^2 = \Theta(N)$, in the multi-instance setting the time/memory/instance tradeoff is $st^2 = \Theta(m^2N)$. This requires slightly larger s, t .

However, whereas in the single instance setting we needed walk lengths t and $2t$ for precomputation and online walks respectively, in the multi-instance setting shorter walks of length t/m and $2t/m$ respectively are sufficient, as we show with the following theorem.

Theorem 2. *Let $(\mathcal{G}, \mathcal{X}, \star)$ be a regular group action with $N = |\mathcal{G}| = |\mathcal{X}|$, and let $y_i = \mathbf{g}^{v_i} \star x$, $1 \leq i \leq m$ be GA-dlog instances. For any choice of $s, t \in \mathbb{N}$ with $4st^2 \leq m^2N$, PRECOMPUTE-MULT-GA (Algorithm 3) precomputes within st/m steps a hint $\mathcal{L}$ of size $\tilde{O}(s)$. Using $\mathcal{L}$, ONLINE-MULT-GA-DLOG (Algorithm 4) runs in a total of $O(t)$ steps, and solves each GA-dlog y_i instance with success probability $\Omega\left(\frac{st^2}{m^2N}\right)$.*

Proof. The proof will closely follow the one for Theorem 1. PRECOMPUTE-MULT-GA computes s walks with t/m steps each, with a total of st/m steps. Storing s endpoints requires memory $\tilde{O}(s)$. Moreover ONLINE-MULT-GA-DLOG performs m walks $\overline{W}^{(1)}, \dots, \overline{W}^{(m)}$ with $2t/m$ steps, that is a total of $\mathcal{O}(t)$ steps. It remains to show the success probability of ONLINE-MULT-GA-DLOG.

Algorithm 3: PRECOMPUTE-MULT-GA

Input: group action parameters $(\mathbf{g}, x) \in \mathcal{G}^n \times \mathcal{X}$, $N := |\mathcal{G}|$, number of instances m , basis for $\Lambda \subseteq \mathbb{Z}^n$, PRF $h : \mathcal{G} \rightarrow \mathbb{Z}^n/\Lambda$

Output: list $\mathcal{L}$ of endpoints/GA-dlogs $(x_{t/m}^{(i)}, \mathbf{w}_{t/m}^{(i)}) \in \mathcal{X} \times \mathbb{Z}^n/\Lambda$

- 1 Choose $s, t \in \mathbb{N}$ s.t. $4st^2 \leq m^2N$ $\triangleright$ E.g. $s = m^2N^{\frac{1}{3}}, t = \frac{1}{2}N^{\frac{1}{3}}$
- 2 $\mathcal{L} \leftarrow \emptyset$
- 3 **for** $i = 1, \dots, s$ *// Compute random walks $W^{(1)}, \dots, W^{(s)}$.*
- 4 **do**
- 5 Choose a random $\mathbf{w}_0^{(i)} \in \mathbb{Z}^n/\Lambda$.
- 6 Let $x_0^{(i)} := \mathbf{g}^{\mathbf{w}_0^{(i)}} \star x \in \mathcal{X}$. *// Randomized starting point.*
- 7 **for** $j = 1, \dots, t/m$ *// Each walk $W^{(i)}$ has length t/m .*
- 8 **do**
- 9 Let $x_j^{(i)} := \mathbf{g}^{h(x_{j-1}^{(i)})} \star x_{j-1}^{(i)} \in \mathcal{X}$ and $\mathbf{w}_j^{(i)} := h(x_{j-1}^{(i)}) + \mathbf{w}_{j-1}^{(i)} \bmod \Lambda$.
- 10 $\mathcal{L} \leftarrow \mathcal{L} \cup \left\{ (x_{t/m}^{(i)}, \mathbf{w}_{t/m}^{(i)}) \right\}$ *// Store endpoint/GA-dlog in hint $\mathcal{L}$.*
- 11 Sort $\mathcal{L}$ by first entry. *// Allows for binary search in $\mathcal{L}$.*
- 12 **return** $\mathcal{L}$

Let X_i be a random variable counting the number of new elements in $\mathcal{X}$ touched by the precomputed random walk W_i , for $i \in \{1, \dots, s\}$. Let $X = \sum_{i=1}^s X_i$. As $X_i \leq \frac{t}{m}$, we have that $X \leq \frac{st}{m}$. Using Bernoulli's inequality and $4st^2 \leq m^2N$, each walk touches the maximum number $X_i = t/m$ of new elements with probability

$$\mathbf{P}[X_i = t/m] \geq \left(\frac{N - \frac{st}{m}}{N} \right)^{t/m} = \left(1 - \frac{st}{mN} \right)^{t/m} \geq 1 - \frac{st^2}{m^2N} \geq \frac{3}{4}.$$

Therefore, we have $\mathbf{E}[X_i] \geq \frac{3}{4} \cdot \frac{t}{m} = \frac{3t}{4m}$ and $\mathbf{E}[X] \geq \frac{3st}{4m}$. By Markov's inequality, we obtain

$$\mathbf{P} \left[X < \frac{st}{2m} \right] \leq \mathbf{P} \left[\frac{st}{m} - X \geq \frac{st}{2m} \right] \leq \frac{\frac{st}{m} - \mathbf{E}[X]}{\frac{st}{2m}} \leq \frac{1}{2}.$$

Let $E^{(i)}$ be the event that online walk $\overline{W}^{(i)}$ collides with some precomputed walk within the first t/m steps. In this case, $\overline{W}^{(i)}$ reaches within the subsequent t/m steps an endpoint in $\mathcal{L}$, thereby solving the GA-dlog instance $y_i = \mathbf{g}^{\mathbf{v}_i} \star x$.

Using $1 - x \leq e^{-x}$ and $1 - e^{-x} \geq x/2$ for $x \leq 1$, we obtain

$$\mathbf{P} \left[E^{(i)} \mid X \geq \frac{st}{2m} \right] \geq 1 - \left(1 - \frac{st}{2mN} \right)^{\frac{t}{m}} \geq 1 - e^{-\frac{st^2}{2m^2N}} \geq \frac{st^2}{4m^2N}.$$

Algorithm 4: ONLINE-MULT-GA-DLOG

Input: $(\mathbf{g}, x, y_1 = \mathbf{g}^{\mathbf{v}_1} \star x, \dots, y_{m'} = \mathbf{g}^{\mathbf{v}_{m'}} \star x) \in \mathcal{G}^n \times \mathcal{X}^{m'+1}, N := |\mathcal{G}|$,
basis of $\Lambda \subseteq \mathbb{Z}^n$, hint $\mathcal{L} \in (\mathcal{X} \times \mathbb{Z}^n / \Lambda)^s$, m, t , PRF $h : \mathcal{X} \rightarrow \mathbb{Z}^n / \Lambda$

Output: GA-dlog $\mathbf{v}_1, \dots, \mathbf{v}_{m'} \in \mathbb{Z}^n / \Lambda$ or FAIL

```

1 for  $i = 1, \dots, m'$  // Initially  $m' = m$ , see also Algorithm 5.
2 do
3   Choose a random  $\mathbf{w}_0 \in \mathbb{Z}^n / \Lambda$ .
4   Let  $x_0 := \mathbf{g}^{\mathbf{w}_0} \star y_i$ . // Start walk  $\overline{W}^{(i)}$  for randomized instance  $y_i$ .
5   for  $j = 1, \dots, 2t/m$  //  $2t/m$ -step walk.
6   do
7     Let  $x_j := \mathbf{g}^{h(x_{j-1})} \star x_{j-1} \in \mathcal{X}$  and  $\mathbf{w}_j := h(x_{j-1}) + \mathbf{w}_{j-1} \bmod \Lambda$ .
8     if  $(x_j, \mathbf{w}_{t/m}^{(\ell)}) \in \mathcal{L}$  for some  $\ell \in \{1, \dots, s\}$  // Endpoint in  $\mathcal{L}$ ?
9     then
10      return  $\mathbf{v}_i := \mathbf{w}_{t/m}^{(\ell)} - \mathbf{w}_j \bmod \Lambda$ 
11  return FAIL

```

Thus, the i -th instance y_i can be solved with probability at least

$$\mathbf{P} \left[E^{(i)} \cap \left(X \geq \frac{st}{2m} \right) \right] \geq \frac{st^2}{8m^2N} = \Omega \left(\frac{st^2}{8m^2N} \right). \quad \square$$

Assume that we choose our parameters such that $st^2 = \Omega(m^2N)$. Then Theorem 2 guarantees constant success probability of ONLINE-MULT-GA-DLOG for each GA-dlog instance $y_i = \mathbf{g}^{\mathbf{v}_i} \star x$ using a single online walk $\overline{W}^{(i)}$. In other words, after running all online walks $\overline{W}^{(1)}, \dots, \overline{W}^{(m)}$ once, we have already solved a constant fraction of all GA-dlogs. We then iterate only with online walks for unsolved instances, until we eventually solve *all* GA-dlogs.

The resulting algorithm ALL-GA-DLOGS is provided as Algorithm 5.

Algorithm 5: ALL-GA-DLOGS

Input: $(\mathbf{g}, x, y_1 = \mathbf{g}^{\mathbf{v}_1} \star x, \dots, y_m = \mathbf{g}^{\mathbf{v}_m} \star x) \in \mathcal{G}^n \times \mathcal{X}^{m+1}, N := |\mathcal{G}|$,
basis of $\Lambda \subseteq \mathbb{Z}^n$, hint $\mathcal{L} \in (\mathcal{X} \times \mathbb{Z}^n / \Lambda)^s$, m, t , PRF $h : \mathcal{X} \rightarrow \mathbb{Z}^n / \Lambda$

Output: all GA-dlog $\mathbf{v}_1, \dots, \mathbf{v}_m \in \mathbb{Z}^n / \Lambda$

```

1 Let  $Y := \{y_1, \dots, y_m\}$ . // Set of unsolved instances.
2 while  $Y \neq \emptyset$  do
3    $Y' \leftarrow \text{ONLINE-MULT-GA-DLOG}(\mathbf{g}, x, Y, N, B(\Lambda), \mathcal{L}, m, t, h)$ 
4   return  $Y'$ . // Output solved instances.
5    $Y := Y \setminus Y'$ . // Remove solved instances.

```

Theorem 3. *Let $(\mathcal{G}, \mathcal{X}, \star)$ be a regular group action with $N = |\mathcal{G}| = |\mathcal{X}|$, and let $y_i = \mathbf{g}^{\mathbf{v}_i} \star x$, $1 \leq i \leq m$ be GA-dlog instances. For any choice $s, t \in \mathbb{N}$ with $4st^2 \leq m^2N$ and $st^2 = \Omega(m^2N)$, PRECOMPUTE-MULT-GA (Algorithm 3) precomputes within st/m steps a hint $\mathcal{L}$ of size $\tilde{O}(s)$. Using $\mathcal{L}$, ALL-GA-DLOGS (Algorithm 5) outputs all GA-dlogs $\mathbf{v}_1, \dots, \mathbf{v}_m$ in an expected number of $O(t)$ steps.*

Proof. Let T_j be the total number of steps in all random walks in the j -th application of ONLINE-MULT-GA-DLOG, and let $T = \sum_{j \geq 1} T_j$ denote the total number of steps of ALL-GA-DLOGS.

By our parameter choice $st^2 = \Omega(m^2N)$ and Theorem 2, in the first iteration of the **while**-loop ONLINE-MULT-GA-DLOG solves each single GA-dlog instance y_i within $T_1 = O(t)$ steps with probability $\Omega\left(\frac{st^2}{m^2N}\right) = \Omega(1)$. Let X_i be an indicator random variable for the event that instance y_i remains unsolved after the first iteration of the **while**-loop of algorithm ALL-GA-DLOGS. Thus, there exists a positive constant $0 < c < 1$, such that $\mathbf{P}[X_i = 0] \geq c$.

Let $X = X_1 + \dots + X_m$ denote the amount of unsolved instances after the first execution of the **while**-loop. Then, we have $\mathbf{E}[X] \leq (1 - c)m$. That is, in the second iteration of the **while**-loop we expect at most a $(1 - c)$ -fraction of unsolved instances. Notice that during the second application of ONLINE-MULT-GA-DLOG we start an expected $\mathbf{E}[|Y|] = \mathbf{E}[X] \leq (1 - c)m$ random walks of length (still) $2t/m$. Hence, the expected number of steps during the second iteration is

$$\mathbf{E}[T_2] \leq \mathbf{E}\left[\frac{2t}{m}X\right] = \frac{2t}{m}\mathbf{E}[X] = \frac{2t}{m}(1 - c)m = O((1 - c)t).$$

In general, we obtain $\mathbf{E}[T_j] = O((1 - c)^{j-1}t)$. Thus, the expected total number of steps is

$$\mathbf{E}[T] = \sum_{j \geq 1} \mathbf{E}[T_j] = O\left(t \cdot \sum_{j \geq 0} (1 - c)^j\right) = O\left(\frac{t}{c}\right) = O(t).$$

□

5 Solving Multiple GA-dlogs (without Precomputation)

Interestingly, our multi-instance GA-dlog algorithms PRECOMPUTE-MULT-GA (Algorithm 3), ONLINE-MULT-GA-DLOG (Algorithm 4) and ALL-GA-DLOG (Algorithm 5) also provide a solution to the multi-instance GA-dlog problem *without precomputation*.

Let $y_1 = \mathbf{g}^{\mathbf{v}_1} \star x, \dots, y_m = \mathbf{g}^{\mathbf{v}_m} \star x$ be m GA-dlog instances. Now apply Theorem 2 with the parameter choice $s = m$ and $t = \frac{1}{2}\sqrt{mN}$. We obtain the following corollary.

Corollary 1. *Let $(\mathcal{G}, \mathcal{X}, \star)$ be a regular group action with $N = |\mathcal{G}| = |\mathcal{X}|$, and let $y_i = \mathbf{g}^{\mathbf{v}_i} \star x$, $1 \leq i \leq m$ be GA-dlog instances. Then PRECOMPUTE-MULT-GA (Algorithm 3) precomputes within $t = O(\sqrt{mN})$ steps a hint $\mathcal{L}$ of size $\tilde{O}(s) = \tilde{O}(m)$. Using $\mathcal{L}$, ONLINE-MULT-GA-DLOG (Algorithm 4) runs in a total of $O(t) = O(\sqrt{mN})$ steps, and solves each GA-dlog instance y_i with constant success probability.*

Notice that our parameter choice balances the run times of the precomputation phase and the online phase. Thus, we can rewrite Corollary 1 more compactly as follows.

Corollary 2. *Let $(\mathcal{G}, \mathcal{X}, \star)$ be a regular group action with $N = |\mathcal{G}| = |\mathcal{X}|$, and let $y_i = \mathbf{g}^{\mathbf{v}_i} \star x$, $1 \leq i \leq m$ be GA-dlog instances. Then one can solve each GA-dlog y_i with constant success probability within a total of $O(\sqrt{mN})$ steps.*

Using the technique of Algorithm 5 and Theorem 3, one obtains the following result.

Corollary 3. *Let $(\mathcal{G}, \mathcal{X}, \star)$ be a regular group action with $N = |\mathcal{G}| = |\mathcal{X}|$, and let $y_i = \mathbf{g}^{\mathbf{v}_i} \star x$, $1 \leq i \leq m$ be GA-dlog instances. Then one can solve all GA-dlog y_i within an expected total of $O(\sqrt{mN})$ steps.*

Multiple GA-dlog Algorithm. Our algorithm behind Corollary 2 computes in a first phase (previously: precomputation) m instance-independent walks $W^{(1)}, \dots, W^{(m)}$. In a second phase (previously: online), we let all instance-dependent walks $\bar{W}^{(i)}$, $1 \leq i \leq m$ collide into the walks $W^{(1)}, \dots, W^{(m)}$, thereby solving all GA-dlog instances y_i .

Intuition of the Achieved $\sqrt{m}$ Speedup. The GHS algorithm can be considered a special case of our aforementioned *multiple GA-dlog algorithm* with $m = 1$. The GHS algorithm lets some instance-independent walk $W^{(1)}$ of length $\sqrt{N}$ collide with an instance-dependent walk of length $\sqrt{N}$. Since both walks have length $\sqrt{N}$ we have $(\sqrt{N})^2$ pairs of elements in $\mathcal{X}$ that can potentially collide, resulting in constant success probability.

Moreover, our multiple GA-dlog algorithm can be considered as running m copies of GHS *simultaneously*. So why do we actually achieve run time $O(\sqrt{mN})$? And why do we achieve a $\sqrt{m}$ speedup over the naive $O(m\sqrt{N})$?

Indeed, running m independent copies of GHS gives us run time $O(m\sqrt{N})$. Our speedup comes from the *simultaneous* instantiation. After the first phase, we have m walks $W^{(1)}, \dots, W^{(m)}$, each of length t/m , visiting a total of (roughly) $m \cdot \frac{t}{m} = t$ elements. For any instance y_i , it suffices that its walk $\bar{W}^{(i)}$ with length roughly t/m collides into *any* of these m walks. This happens with constant probability if the product of visited elements in $\mathcal{X}$ in *all* instance-independent walks with the instance-dependent walk length roughly equals N . More precisely, we require

$$\left(m \cdot \frac{t}{m}\right) \cdot \frac{t}{m} = \Theta(N).$$

Solving for run time t yields $t = \Theta(\sqrt{mN})$.

6 Classical Multiple Dlogs (without Precomputation)

The algorithm underlying Corollary 2 also applies in the classical multi-instance dlog setting [8], as well as for other multi-instance settings [13,23].

We provide the *multi-instance dlog algorithm* for completeness in Algorithm 6, since this setting is of great importance in cryptography. The algorithm is appealingly simple and allows for a clean analysis.

Algorithm 6: MULT-DLOG

Input: $(g, y_1 = g^{v_1}, \dots, y_m = g^{v_m}) \in G^{m+1}$, $N := |G|$, PRF $h : G \rightarrow \mathbb{Z}_N$
Output: all dlogs $v_1, \dots, v_m \in \mathbb{Z}_N$

```

1  $\mathcal{L} \leftarrow \emptyset$ 
2 for  $i = 1, \dots, m$  // Compute random walks  $W^{(1)}, \dots, W^{(m)}$ .
3 do
4   Choose a random  $w_0^{(i)} \in \mathbb{Z}_N$ .
5   Let  $x_0^{(i)} := g^{w_0^{(i)}} \in G$ . // Randomized starting point.
6   for  $j = 1, \dots, t/m$  // Each walk  $W^{(i)}$  has length  $t/m$ .
7   do
8     Let  $x_j^{(i)} := g^{h(x_{j-1}^{(i)})} \cdot x_{j-1}^{(i)} \in G$  and  $w_j^{(i)} := h(x_{j-1}^{(i)}) + w_{j-1}^{(i)} \bmod N$ .
9      $\mathcal{L} \leftarrow \mathcal{L} \cup \{(x_{t/m}^{(i)}, w_{t/m}^{(i)})\}$  // Store endpoint/dlog in  $\mathcal{L}$ .
10 Sort  $\mathcal{L}$  by first entry. // Allows for binary search in  $\mathcal{L}$ .
11 for  $i = 1, \dots, m$  do
12   while instance  $y_i$  is unsolved do
13     Let  $w_0 \in_R \mathbb{Z}_N$  and  $x_0 := g^{w_0} \cdot y_i$ . // Walk  $\overline{W}^{(i)}$  for instance  $y_i$ .
14     for  $j = 1, \dots, 2t/m$  //  $2t/m$ -step walk
15     do
16       Let  $x_j := g^{h(x_{j-1})} \cdot x_{j-1} \in G$  and  $w_j := h(x_{j-1}) + w_{j-1} \bmod N$ .
17       if  $(x_j, w_{t/m}^{(\ell)}) \in \mathcal{L}$  for some  $\ell \in \{1, \dots, m\}$  // Endpoint in  $\mathcal{L}$ ?
18       then
19         return  $v_\ell := w_{t/m}^{(\ell)} - w_j \bmod N$ 
```

Let $(G, \cdot) = \langle g \rangle$ be a finite abelian group with $N = |G|$. Let $y_1 = g^{v_1}, \dots, y_m = g^{v_m}$ be m dlog instances with $v_i \in \mathbb{Z}_N$. Then MULT-DLOG (Algorithm 6) describes an algorithm for solving all dlog instances.

Theorem 4. *Let $(G, \cdot) = \langle g \rangle$ be an abelian group with order $N = |G|$, and let $y_i = g^{v_i}$, $1 \leq i \leq m$ be dlog instances. Then MULT-DLOG (Algorithm 6) outputs within an expected $O(\sqrt{mN})$ steps all dlogs $v_i \in \mathbb{Z}_N$.*

Proof. The proof is a special case of the proofs of Theorem 2 and Theorem 3 with the parameter choice $s = m$ and $t = \frac{1}{2}\sqrt{mN}$. Theorem 2 guarantees for this parameter choice for every dlog instance y_i constant success probability $c = \Omega(1)$. Therefore, we solve every instance y_i after an expected $\frac{1}{c} = O(1)$ re-randomized runs of $\overline{W}^{(i)}$. $\square$

7 Experimental Results for CSIDH

In this section, we instantiate our precomputation algorithm from Section 3 for the prominent group action based post-quantum scheme CSIDH.

Recall that, by Definition 1, for a group $\mathcal{G}$ and a set $\mathcal{X}$ a group action is given by a mapping $\star : \mathcal{G} \times \mathcal{X} \rightarrow \mathcal{X}$. Moreover, a group action is called *regular* if $\mathcal{X} = \{g \star x \mid g \in \mathcal{G}\}$ and $|\mathcal{X}| = |\mathcal{G}|$. In order to instantiate our algorithms, we first need to define $(\mathcal{G}, \mathcal{X}, \star)$ for CSIDH.

The Group $\mathcal{G}$ in CSIDH. Let E be an elliptic curve defined over $\mathbb{F}_p$, for some prime number $p \geq 5$, and let $\mathcal{O}$ denote the $\mathbb{F}_p$ -rational endomorphism ring of E . We say that E is *supersingular* if it satisfies $|E(\mathbb{F}_p)| = p + 1$. We choose the prime p such that $p + 1 = 4 \cdot \prod_{i=1}^n \ell_i$, where $\ell_1 < \dots < \ell_n$ are small odd primes and $\ell_1 = 3$. This property ensures that the ideal $\ell_i \mathcal{O}$ splits as the product of two prime ideals $\mathfrak{l}_i$ and $\overline{\mathfrak{l}}_i$, namely $\ell_i \mathcal{O} = \mathfrak{l}_i \overline{\mathfrak{l}}_i$. Moreover, let us denote by $\text{Cl}(\mathcal{O})$ the class group of $\mathcal{O}$, and by $[\mathfrak{a}] \in \text{Cl}(\mathcal{O})$ the equivalence class of $\mathfrak{a}$ in $\text{Cl}(\mathcal{O})$, see [10] for details.

We now define the (sub)group $\mathcal{G} = \langle \mathbf{g} \rangle \subseteq \text{Cl}(\mathcal{O})$, where $\mathbf{g} = \{[\mathfrak{l}_1], \dots, [\mathfrak{l}_n]\}$. According to a heuristic from [30], for a given prime p of this form, the size of the group $\mathcal{G}$ is approximately $N \approx p^{1/2}$.

The Set $\mathcal{X}$ and the Action $\star$ in CSIDH. Up to isomorphism, every supersingular elliptic curve over $\mathbb{F}_p$ is defined by an equation

$$Y^2 = X^3 + AX^2 + X,$$

where $A \in \mathbb{F}_p$ is called *Montgomery coefficient*. As shown in [5], supersingular elliptic curves can be uniquely represented (up to isomorphism) by their Montgomery coefficient A . Let $\mathcal{M} \subseteq \mathbb{F}_p$ be the set of all Montgomery coefficients of supersingular curves over $\mathbb{F}_p$. By our choice of the prime p , we have that the curve with equation $Y^2 = X^3 + X$ is supersingular with Montgomery coefficient $A = 0 \in \mathcal{M}$. We define $x = 0 \in \mathcal{M}$ as the origin of $\mathcal{X}$.

It is known that the class group $\text{Cl}(\mathcal{O})$ acts regularly on $\mathcal{M}$, see [12,31]. Let us denote this action by $\star : \text{Cl}(\mathcal{O}) \times \mathcal{M} \rightarrow \mathcal{M}$, and let us define the set $\mathcal{X} = \{g \star x \mid g \in \mathcal{G}\}$. This implies regularity of the action $\star : \mathcal{G} \times \mathcal{X} \rightarrow \mathcal{X}$ restricted to $\mathcal{G} \subseteq \text{Cl}(\mathcal{O})$ and $\mathcal{X} \subseteq \mathcal{M} \subseteq \mathbb{F}_p$.

We now have a well-defined regular group action $(\mathcal{G}, \mathcal{X}, \star)$ with origin $x = 0$ and generators $\mathbf{g} = \{[\mathfrak{l}_1], \dots, [\mathfrak{l}_n]\}$. As in Definition 2, we have representations for the group $\mathcal{G}$, namely for $\mathbf{v} = (v_1, \dots, v_n) \in \mathbb{Z}^n$ we denote $\mathbf{g}^{\mathbf{v}} = \prod_{i=1}^n [\mathfrak{l}_i]^{v_i}$.

Admissible Representations. Let $\mathbf{v} \in \mathbb{Z}^n$ be a representation. The complexity for evaluating the action of the group element $\mathbf{g}^{\mathbf{v}}$ scales proportionally with $\|\mathbf{v}\|_1$, see [1, 33]. Therefore, only group elements with small 1-norm representation can be efficiently computed via the action. For this reason, CSIDH only uses representations within the set $\mathcal{R} = \{-d, \dots, d\}^n$, for some integer $d \in \mathbb{N}$, see [5]. We say that a representation $\mathbf{v}$ is *admissible* if $\mathbf{v} \in \mathcal{R}$. In practice, CSIDH parameters require the inequality $(2d+1)^n \geq N$ to ensure that $\{-d, \dots, d\}^n$ properly covers a large portion of $\mathcal{G}$. The smallest d satisfying this inequality is $d = \left\lceil \frac{\sqrt[n]{N}-1}{2} \right\rceil$.

Algorithm 7: CSIDH $h_{a,b}$

Input: Montgomery coefficient $A \in \mathcal{X} \subseteq \mathbb{F}_p, a, b \in \mathbb{F}_p, p, d, n \in \mathbb{N}$
Output: admissible CSIDH representation $\mathbf{v} \in \mathbb{Z}_{2d+1}^n$

- 1 Let $y := aA + b \bmod p$. // Re-randomize A .
- 2 Expand y in base $(2d+1)$ as $y = \sum_{i=0}^{2n-1} y_i(2d+1)^i$ with $y_i \in \mathbb{Z}_{2d+1}$.
- 3 **for** $i = 0, \dots, n-1$ **do**
- 4 $\lfloor$ Let $v_i := y_i - y_{i+n} \bmod 2d+1$.
- 5 **return** $\mathbf{v} = (v_1, \dots, v_n) \in \mathbb{Z}_{2d+1}^n$

Instantiating a Random Function for CSIDH. Let us identify $\{-d, \dots, d\}$ with $\mathbb{Z}_{2d+1}$. For instantiating our algorithm from Section 3 in the CSIDH setting we need a function $h : \mathcal{X} \rightarrow \mathbb{Z}_{2d+1}^n$, where $\mathcal{X} \subseteq \mathbb{F}_p$ and $p^{\frac{1}{2}} \approx N = |\mathcal{X}| \leq (2d+1)^n$.

We provide our function $h_{a,b}$ in Algorithm 7 for some $a, b \in \mathbb{F}_p$ chosen uniformly at random. The purpose of a, b is to re-randomize elements from $\mathcal{X}$ in $\mathbb{F}_p$. Clearly, our function is not a PRF in the cryptographic sense, but our experiments indicate that its randomness properties are sufficient for practical purposes.

Experiments. We implemented our GA-dlog algorithms PRECOMPUTE-GA (Algorithm 1) and ONLINE-GA-DLOG (Algorithm 2) from Section 3 together with the function $h_{a,b}$ (Algorithm 7). Our code is available at https://github.com/maxostuzzi/precomputation_attack.

In our implementation, we slightly deviate from the description of ONLINE-GA-DLOG by restarting an online walk with a re-randomized starting point if it fails to collide into an endpoint after $2t$ steps, see also Remark 1 and Algorithm 6. Our algorithms are instantiated with the parameter choice $s, t = N^{\frac{1}{3}}$.

We applied our algorithm ONLINE-GA-DLOG on ten different primes p . For each prime p , we ran ten GA-dlog instances $y_1, \dots, y_{10}$, until all of them were successfully solved. In Table 1, we list our CSIDH primes p with their respective small odd primes ℓ_i , where $p+1 = 4 \prod_i \ell_i$.

Table 1: CSIDH instances and average number of random walk.

p	ℓ_i small odd primes	# of runs
1019	3, 5, 17	1.3
78539	3, 5, 7, 11, 17	1.0
1021019	3, 5, 7, 11, 13, 17	1.2
19399379	3, 5, 7, 11, 13, 17, 19	1.0
1450388939	3, 5, 7, 11, 13, 19, 31, 41	1.0
53664390779	3, 5, 7, 11, 13, 19, 31, 37, 41	1.0
8575569646643	3, 7, 11, 13, 17, 19, 31, 37, 41, 47	1.1
454505191272131	3, 7, 11, 13, 17, 19, 31, 37, 41, 47, 53	1.2
26815806285055787	3, 7, 11, 13, 17, 19, 31, 37, 41, 47, 53, 59	1.1
138624083338000259	3, 5, 7, 11, 13, 17, 19, 31, 37, 41, 47, 53, 61	1.1

For every prime p , we measured the number of random walks of length $2t$ we had to perform for $y_1, \dots, y_{10}$, and then averaged the number over all 10 instances.

From Table 1, we see that all averages are close to 1. In fact, out of our total of 100 solved instances, 91 were solved by running a single $2t$ -step random walk, indicating a large success probability per random walk. Our experiments also clearly show that our function $h_{a,b}$ (Algorithm 7) provides sufficient randomness in practice.

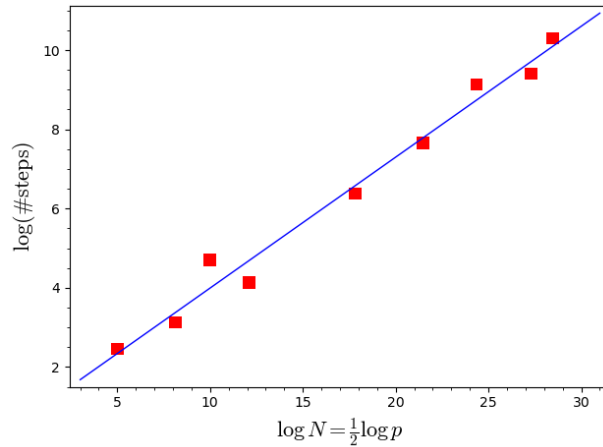


Fig. 3: Performance of ONLINE-GA-DLOG on CSIDH. For every of our 10 choices for p , we average the number of steps over 10 solved instances.

Figure 3 shows the logarithm of the number of steps (as a function of $\log N = \frac{1}{2} \log p$) that ONLINE-GA-DLOG performed until it successfully recovered a GA-dlog, including potential restarts of a walk.

Again, for every p we averaged the number of steps over all 10 solved instances. Despite the re-randomization, which clearly increases the number of steps in the online phase, the slope of the fitting line for our experiments is 0.33, as expected.

Acknowledgments. Alexander May is supported by DFG under Germany’s Excellence Strategy - EXC 2092 CASA - 390781972. Massimo Ostuzzi is part of the Quantum-Safe Internet (QSI) ITN, which received funding from the European Union’s Horizon-Europe programme as Marie Skłodowska-Curie Action (PROJECT 101072637 - HORIZON - MSCA-2021-DN-01). The authors would like to thank Steven Galbraith and the anonymous reviewers for helpful comments.

Disclosure of Interests. The authors have no competing interests.

References

1. Bernstein, D.J., Lange, T.: Computing small discrete logarithms faster. pp. 317–338 (2012). https://doi.org/10.1007/978-3-642-34931-7_19
2. Beullens, W., Kleinjung, T., Vercauteren, F.: CSI-FiSh: Efficient isogeny based signatures through class group computations. pp. 227–247 (2019). https://doi.org/10.1007/978-3-030-34578-5_9
3. Bonnetain, X., Schrottenloher, A.: Quantum security analysis of CSIDH. pp. 493–522 (2020). https://doi.org/10.1007/978-3-030-45724-2_17
4. Brassard, G., Yung, M.: One-way group actions. pp. 94–107 (1991). https://doi.org/10.1007/3-540-38424-3_7
5. Castryck, W., Lange, T., Martindale, C., Panny, L., Renes, J.: CSIDH: An efficient post-quantum commutative group action. pp. 395–427 (2018). https://doi.org/10.1007/978-3-030-03332-3_15
6. Chávez-Saab, J., Chi-Domínguez, J.J., Jaques, S., Rodríguez-Henríquez, F.: The SQALE of CSIDH: sublinear Vélu quantum-resistant isogeny action with low exponents. *Journal of Cryptographic Engineering* **12**(3), 349–368 (2022)
7. Childs, A., Jao, D., Soukharev, V.: Constructing elliptic curve isogenies in quantum subexponential time. *Journal of Mathematical Cryptology* **8**(1), 1–29 (2014)
8. Corrigan-Gibbs, H., Kogan, D.: The discrete-logarithm problem with preprocessing. pp. 415–447 (2018). https://doi.org/10.1007/978-3-319-78375-8_14
9. Couveignes, J.M.: Hard homogeneous spaces. *Cryptology ePrint Archive*, Report 2006/291 (2006), <https://eprint.iacr.org/2006/291>
10. Cox, D.A.: Primes of the Form $x^2 + ny^2$: Fermat, Class Field Theory, and Complex Multiplication with Solutions, vol. 387. American Mathematical Soc. (2022)
11. De Feo, L., Kieffer, J., Smith, B.: Towards practical key exchange from ordinary isogeny graphs. pp. 365–394 (2018). https://doi.org/10.1007/978-3-030-03332-3_14

12. Delfs, C., Galbraith, S.D.: Computing isogenies between supersingular elliptic curves over $\mathbb{F}_p$ **78**(2), 425–440 (2016). <https://doi.org/10.1007/s10623-014-0010-1>
13. Fouque, P.A., Joux, A., Mavromati, C.: Multi-user collisions: Applications to discrete logarithm, Even-Mansour and PRINCE. pp. 420–438 (2014). https://doi.org/10.1007/978-3-662-45611-8_22
14. Galbraith, S., Stolbunov, A.: Improved algorithm for the isogeny problem for ordinary elliptic curves. *Applicable Algebra in Engineering, Communication and Computing* **24**(2), 107–131 (2013)
15. Galbraith, S.D., Hess, F., Smart, N.P.: Extending the GHS Weil descent attack. pp. 29–44 (2002). https://doi.org/10.1007/3-540-46035-7_3
16. Hafner, J.L., McCurley, K.S.: A rigorous subexponential algorithm for computation of class groups. *Journal of the American mathematical society* **2**(4), 837–850 (1989)
17. Hallgren, S.: Fast quantum algorithms for computing the unit group and class group of a number field. In: *Proceedings of the thirty-seventh annual ACM symposium on Theory of computing*. pp. 468–474 (2005)
18. Heninger, N., Durumeric, Z., Wustrow, E., Halderman, J.A.: Mining your ps and qs: Detection of widespread weak keys in network devices. pp. 205–220 (2012)
19. Jao, D., Azarderakhsh, R., Campagna, M., Costello, C., De Feo, L., Hess, B., Jalali, A., Koziel, B., LaMacchia, B., Longa, P., Naehrig, M., Renes, J., Soukharev, V., Urbanik, D., Pereira, G., Karabina, K., Hutchinson, A.: SIKE. Tech. rep., National Institute of Standards and Technology (2022), available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/round-4-submissions>
20. Jao, D., De Feo, L.: Towards quantum-resistant cryptosystems from supersingular elliptic curve isogenies. pp. 19–34 (2011). https://doi.org/10.1007/978-3-642-25405-5_2
21. Kuhn, F., Struik, R.: Random walks revisited: Extensions of Pollard’s rho algorithm for computing multiple discrete logarithms. pp. 212–229 (2001). https://doi.org/10.1007/3-540-45537-X_17
22. Kuperberg, G.: A subexponential-time quantum algorithm for the dihedral hidden subgroup problem. *SIAM Journal on Computing* **35**(1), 170–188 (2005)
23. May, A., Zwegdinger, F.: Legendre PRF (multiple) key attacks and the power of preprocessing. pp. 428–438 (2022). <https://doi.org/10.1109/CSF54842.2022.9919640>
24. Panny, L., Petit, C., Stopar, M.: KLaPoTi: An asymptotically efficient isogeny group action from 2-dimensional isogenies. *Cryptology ePrint Archive, Paper 2024/1844* (2024), <https://eprint.iacr.org/2024/1844>
25. Peikert, C.: He gives C-sieves on the CSIDH. pp. 463–492 (2020). https://doi.org/10.1007/978-3-030-45724-2_16
26. Pohlig, S., Hellman, M.: An improved algorithm for computing logarithms over $GF(p)$ and its cryptographic significance. *IEEE Transactions on Information Theory* **24**(1), 106–110 (1978)
27. Pollard, J.M.: Monte Carlo methods for index computation. *Mathematics of computation* **32**(143), 918–924 (1978)
28. Schoof, R.: Counting points on elliptic curves over finite fields. *Journal de théorie des nombres de Bordeaux* **7**(1), 219–254 (1995)
29. Shor, P.W.: Algorithms for quantum computation: Discrete logarithms and factoring. pp. 124–134 (1994). <https://doi.org/10.1109/SFCS.1994.365700>
30. Siegel, C.: Über die Classenzahl quadratischer Zahlkörper. *Acta Arithmetica* **1**(1), 83–86 (1935)

31. Silverman, J.H.: Advanced topics in the arithmetic of elliptic curves, vol. 151. Springer Science & Business Media (1994)
32. Stolbunov, A.: Constructing public-key cryptographic schemes based on class group action on a set of isogenous elliptic curves. *Adv. Math. Commun.* **4**(2), 215–235 (2010)
33. V  lu, J.: Isog  nies entre courbes elliptiques. *Comptes-Rendus de l’Acad  mie des Sciences* **273**, 238–241 (1971)
34. Yun, A.: Generic hardness of the multiple discrete logarithm problem. pp. 817–836 (2015). https://doi.org/10.1007/978-3-662-46803-6_27